

Сети (карты) Кохонена. Сеть встречного распространения.

Кластеризация - это разделение исследуемого множества объектов на группы "похожих" объектов, называемых кластерами.

Решением задачи классификации является отнесение каждого из объектов к одному из заранее определенных классов. В задаче кластеризации происходит отнесение объекта к одному из заранее неопределенных классов. Разбиение объектов по кластерам осуществляется при одновременном формировании кластеров.

Формально задача кластеризации описывается следующим образом :

Дано множество объектов $I = \{i_1, i_2, i_3 \dots i_n\}$, каждый из которых характеризуется вектором x_j , $j = 1, 2, \dots, n$ атрибутов (параметров): $x_j = \{x_{j1}, x_{j2}, x_{j3}, x_{j4} \dots x_{jm}\}$

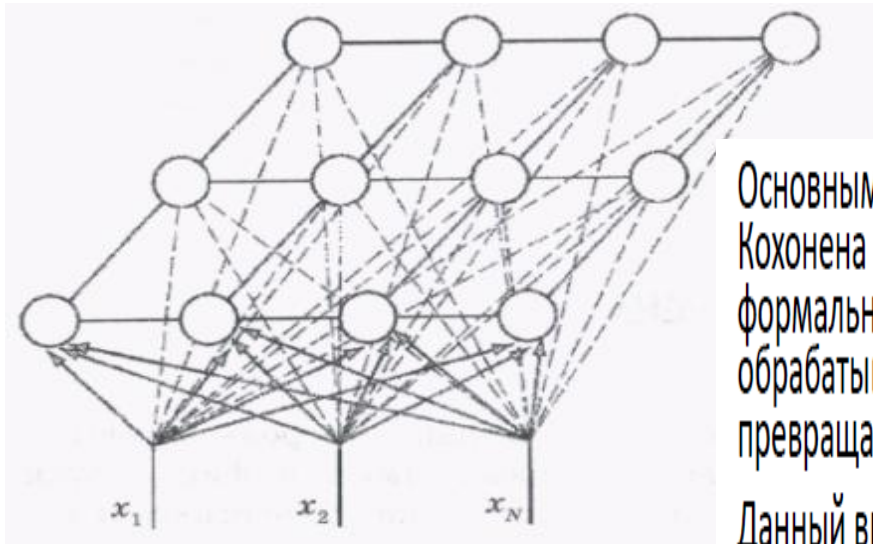
Требуется построить множество кластеров C и отображение F множества I на множество C , то есть $F : I \rightarrow C$.

Задача кластеризации состоит в построении множества $C = \{c_1, c_2, c_3, \dots, c_k, \dots, c_g\}$, где c_k — кластер, содержащий "похожие" объекты из множества I : $c_k = \{i_j, i_p | i_j \in I, i_p \in I, \text{ и } d(i_j, i_p) < \sigma\}$,

σ — величина, определяющая меру близости для включения объектов в один кластер,

$d(i_j, i_p)$ — мера близости между объектами, называемая расстоянием. Если расстояние $d(i_j, i_p)$ меньше некоторого значения σ , то объекты считаются близкими и помещаются в один кластер.

- ▶ Искусственная нейронная **сеть Кохонена** или самоорганизующаяся карта была предложена финским исследователем Тойво Кохоненом в начале 1980-х годов.
- ▶ Она представляет собой двухслойную сеть. Каждый нейрон первого (распределительного) слоя соединен со всеми нейронами второго (выходного) слоя, которые расположены в виде двумерной решетки.
- ▶ Нейроны выходного слоя называются кластерными элементами, их количество определяют максимальное количество групп, на которые система может разделить входные данные. Увеличивая количество нейронов второго слоя можно увеличивать детализацию результатов процесса кластеризации.



Основным элементом данного вида сетей является слой Кохонена. Слой Кохонена состоит из адаптивных линейных сумматоров («линейных формальных нейронов»). Как правило, выходные сигналы слоя Кохонена обрабатываются по правилу «победитель забирает всё»: наибольший сигнал превращается в единичный, остальные обращаются в ноль.

Данный вид нейронных сетей относится к самоорганизующимся нейронным сетям и позволяет решать такие задачи как: моделирование, прогнозирование, поиск закономерностей в больших массивах данных, выявление наборов независимых признаков и сжатие информации.

Функционирование сети

Система работает по принципу соревнования – нейроны второго слоя соревнуются друг с другом за право наилучшим образом сочетаться с входным вектором сигналов, побеждает тот элемент – нейрон, чей вектор весов ближе всего к входному вектору сигналов. За меру близости двух векторов можно взять квадрат евклидова расстояния. Таким образом, каждый входной вектор относится к некоторому кластерному элементу.

Алгоритм обучения сети Кохонена

Вход: обучающая выборка, темп обучения α_0 (обычно 0.7 - 0.9).

Выход: веса нейронов Кохонена $w_{i,j}$

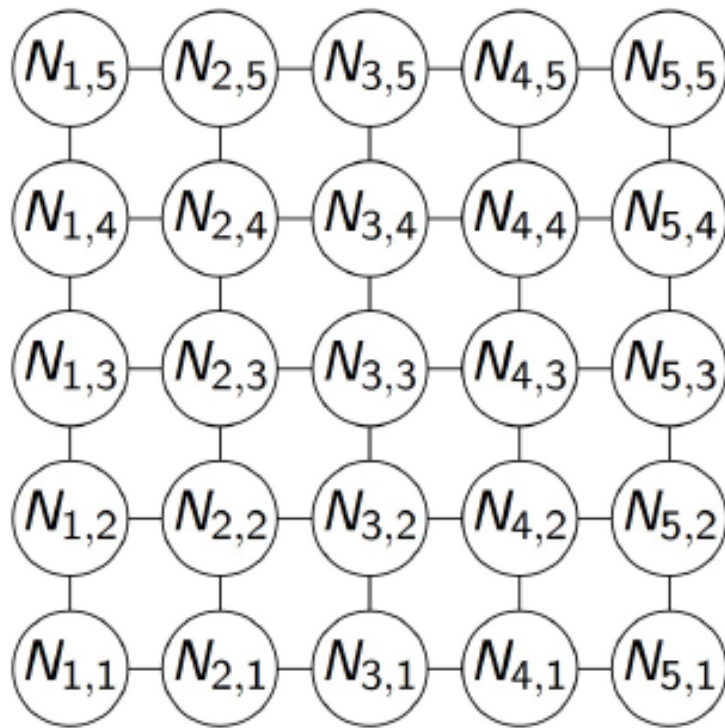
1. задать случайно значения весов;
2. **повторять**
3. для данного входного сигнала x найти нейрон-победитель;
4. скорректировать веса победителя (WTA) или победителя и его соседей (WTM)

$$w_{i,j} = w_{i,j} + \alpha_N(x - w_{i,j});$$

5. уменьшить темп обучения ($\alpha_{N+1} = \alpha_N - 0.0001$);
6. **пока** w не стабилизируются.

Правило жесткой конкуренции WTA (winner takes all)

Пусть $X \in R^n$ – пространство объектов, которые нужно разнести по M кластерам. Инициализируем начальные веса $w_{i,j} \in R^n$ нейронов $N_{i,j}$.



Подадим
на вход сети объект $x \in X$.
Побеждает тот нейрон,
чьи веса меньше всего
отличаются от координат x .

$$\|x - w_{i,j}\| \rightarrow \min$$

Нейрон-победитель обучается
по правилу WTA (winner takes
all): $w_{i,j} = w_{i,j} + \eta(x - w_{i,j})$.

Правило мягкой конкуренции WTM (winner takes most)

Будем обучать все нейроны, но для каждого будет свой шаг обучения:

$$\eta_{k,l} = \eta \rho^{|k-i|+|l-j|}, \quad \rho < 1.$$

Правило обучения WTM (winner takes most):

$$w_{k,l} = w_{k,l} + \eta_{k,l}(x - w_{k,l}).$$

На каждой итерации все нейроны смещаются в сторону x . Смещение тем больше, чем ближе нейрон находится к нейрону-победителю.

Метод выпуклой комбинации

При использовании WTA некоторые нейроны Кохонена при случайной инициализации весов могут не побеждать вообще и оказаться бесполезными.

Метод выпуклой комбинации (convex combination method):

- вначале веса и входные векторы близки:

$$w = \frac{1}{\sqrt{n}},$$

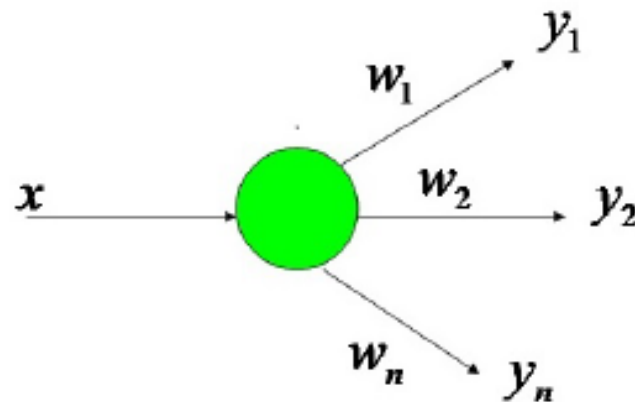
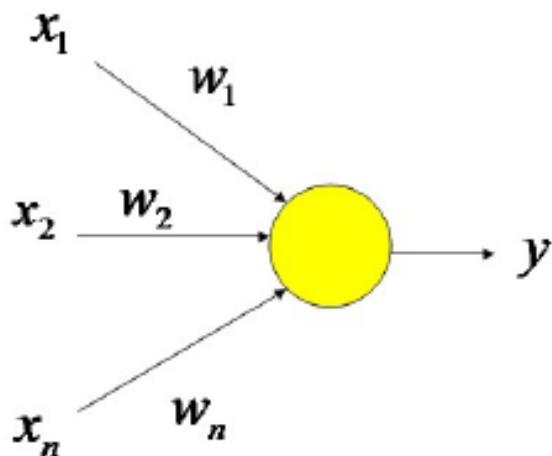
$$x = ax + \frac{1-a}{\sqrt{n}} \approx \frac{1}{\sqrt{n}} \text{ при } a \approx 0$$

- затем a постепенно возрастает ($a \rightarrow 1$) и входные векторы принимают свои истинные значения x . Весовые векторы при этом отслеживают один или группу входных векторов и следуют за ними.

Входная и выходная звезда Гроссберга

Входная звезда (instar) Гроссберга - нейрон с n входами x_i и весами w_i и одним выходом $y = \sum_{i=1}^n x_i w_i$.

Выходная звезда (outstar) Гроссберга - нейрон с одним входом x и n выходами y_i с весами w_i .



Instar learning

Входная звезда обучается реагировать на определенный входной вектор x (или на вектор определенного класса). Если веса и входные векторы нормированы, то

$$y = \sum_{i=1}^n x_i w_i$$

будет максимальным для «своего» входного образа. В процессе обучения веса корректируются:

$$w = w + \alpha(x - w),$$

α – темп обучения. Обычно $\alpha = 0.1$ и уменьшается в процессе обучения.

Outstar learning

Выходная звезда обучается выдавать требуемый выход при поступлении сигнала на вход.

Формула коррекции весов:

$$w = w + \beta(y - w),$$

β - темп обучения. Обычно $\beta \approx 1$ и уменьшается до нуля в процессе обучения.

Алгоритм обучения слоя Гроссберга

Вход: пара x, y из обучающей выборки (вход и выход), темп обучения β_0 (обычно 0.1);

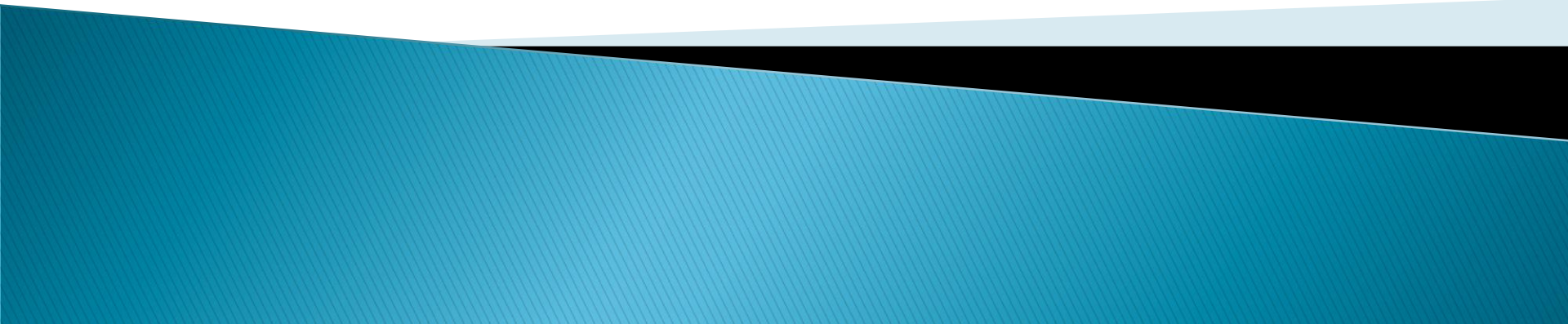
Выход: веса нейронов Гроссберга $v_{i,j}$

1. задать случайно значения весов;
2. **повторять**
3. для данного входного сигнала x найти нейрон-победитель слоя Кохонена;
4. скорректировать веса слоя Гроссберга, связанные с выигравшим i -ым нейроном Кохонена

$$v_{i,j} = v_{i,j} + \beta_N(y_j - v_{i,j});$$

5. уменьшить темп обучения ($\beta_{N+1} = \beta_N - 0.0001$);
6. **пока** v не стабилизируются.

Сети встречного распространения (The counterpropagation network CPN)

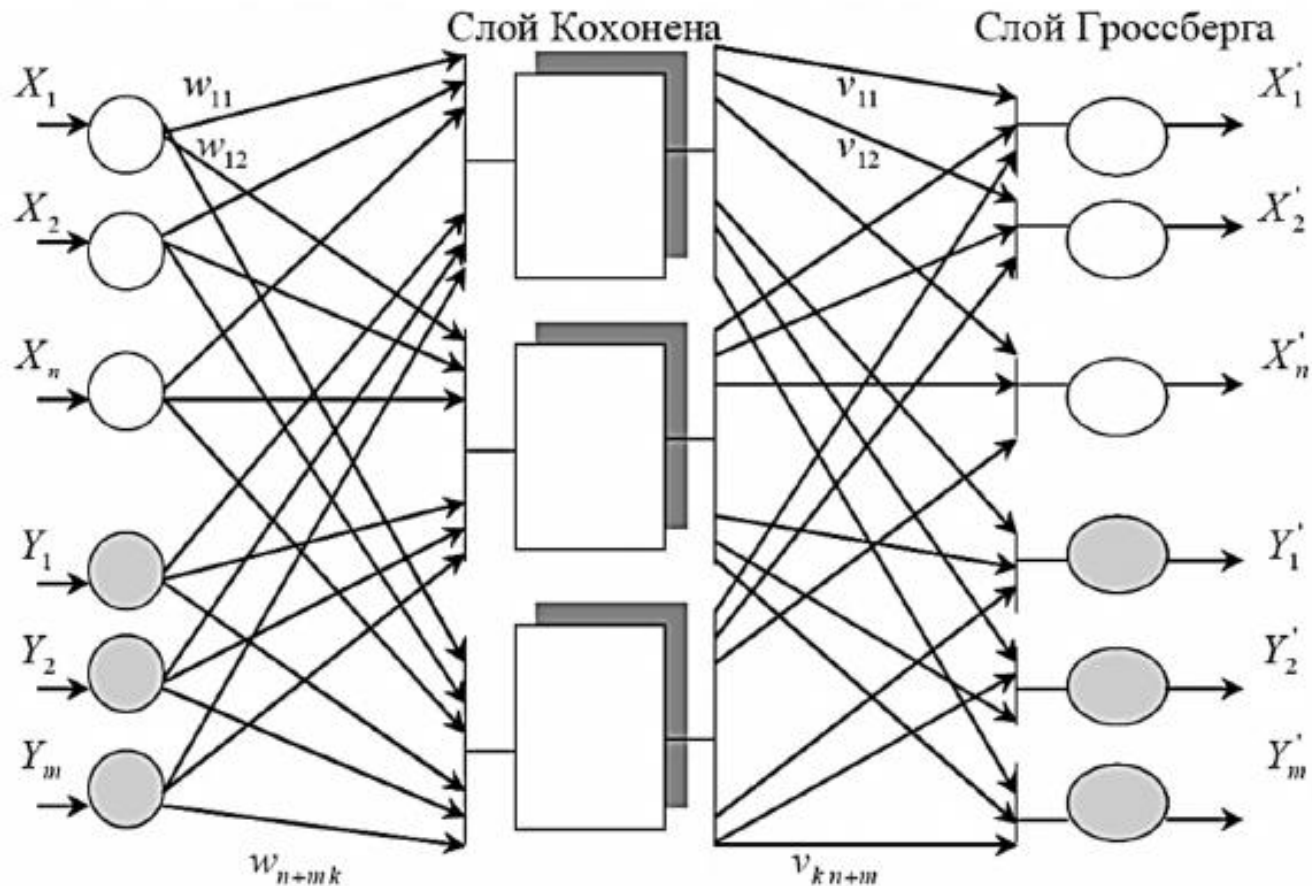




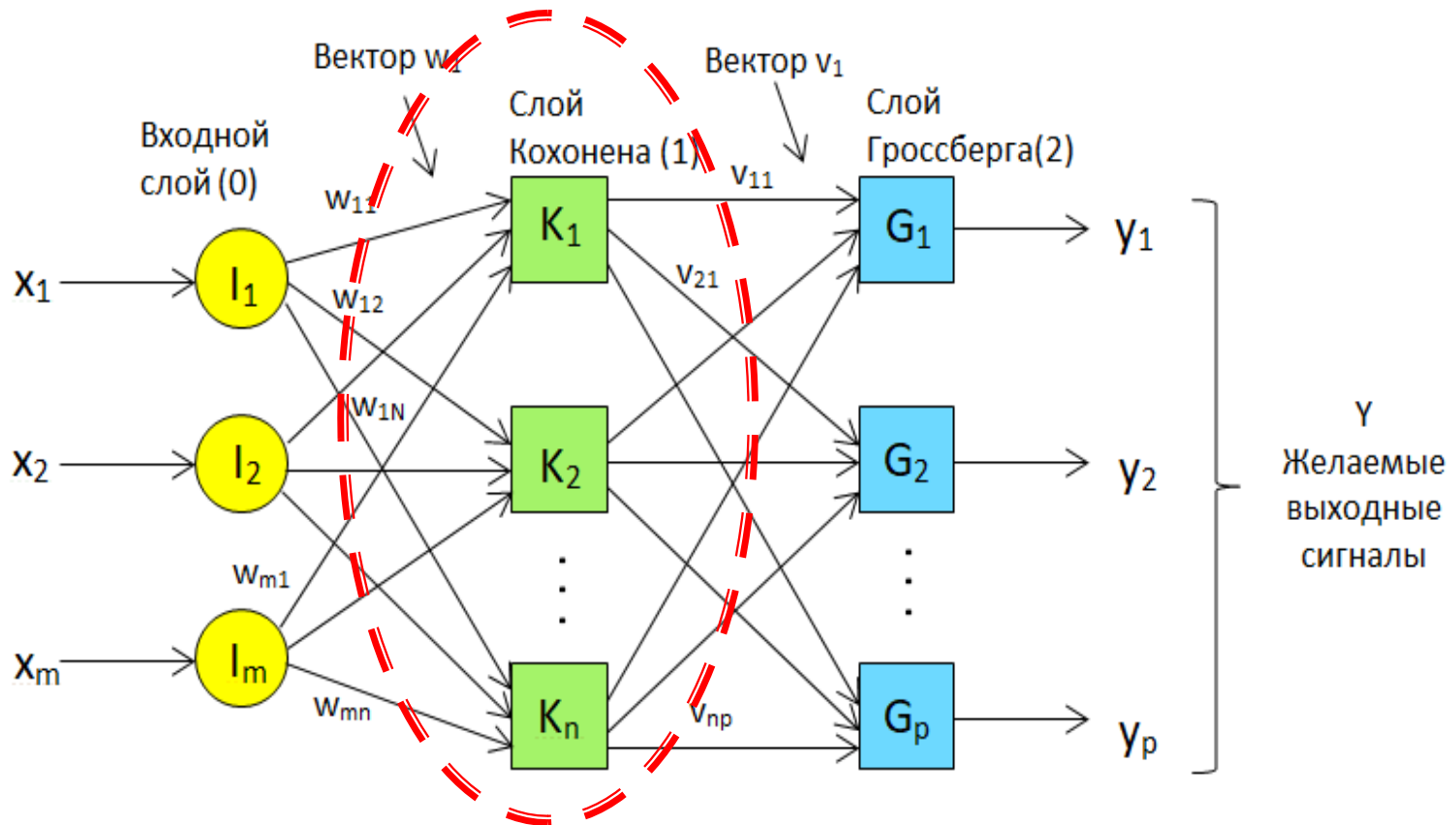
- Автором сетей встречного распространения является профессор Калифорнийского университета в Сан-Диего Роберт Хехт-Нильсон.
- Сети данного вида объединяют 2 хорошо известных алгоритма:
 1. Самоорганизующиеся карты Кохонена
 2. Выходная звезда Гроссберга

Двунаправленная сеть встречного распространения (Full CPN)

позволяет не только по входу находить выход, но и наоборот.

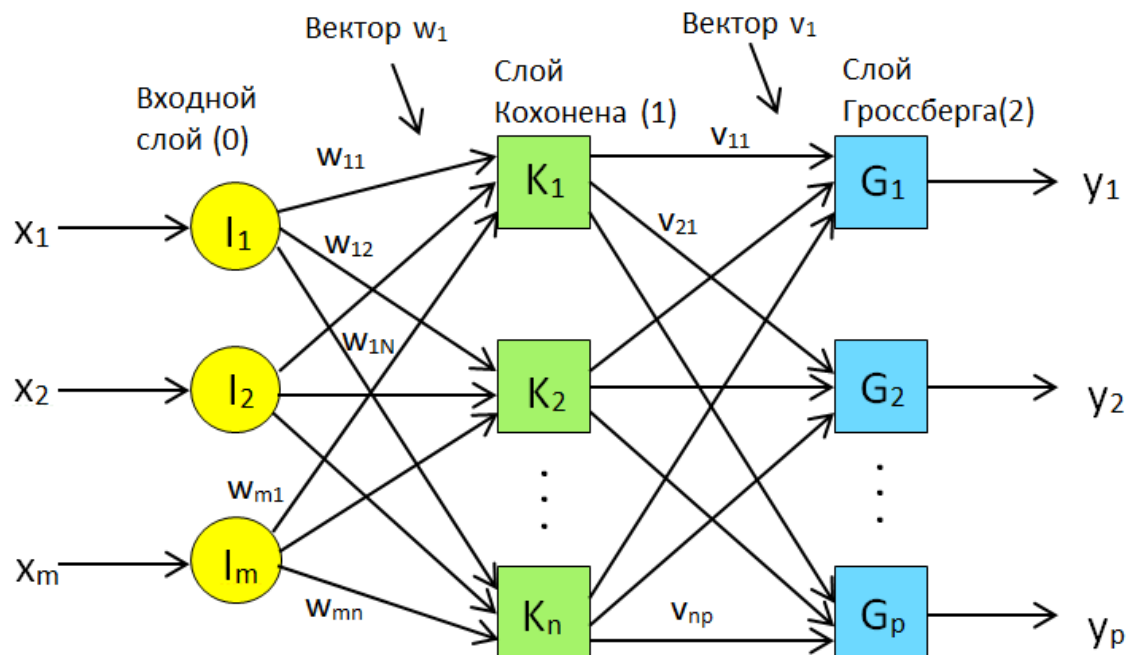


Слой Кохонена



В своей простейшей форме слой Кохонена функционирует в духе "победитель забирает все", т.е. для данного входного вектора один и только один нейрон Кохонена выдает на выходе логическую единицу, а все остальные выдают ноль.

Слой Кохонена



- $NET_j = \sum_i x_i w_{ij}$,
где NET_j – это выход NET нейрона Кохонена j , или в векторной записи,
- $N = XW$, где N – вектор выходов NET слоя кохонена.

Нейрон Кохонена с максимальным значением NET является "победителем". Его выход равен единице, у остальных он равен нулю.

Обучение слоя Кохонена

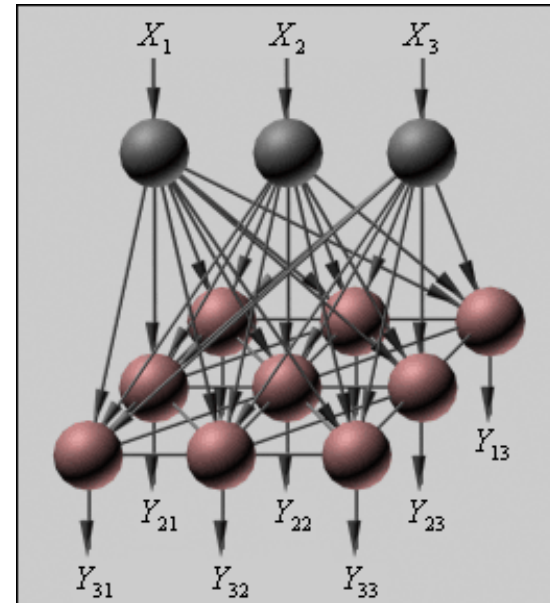
- **Цель:**

классифицировать входные векторы в группы схожих.

- Обучение сети Кохонена является самообучением, протекающим без учителя.

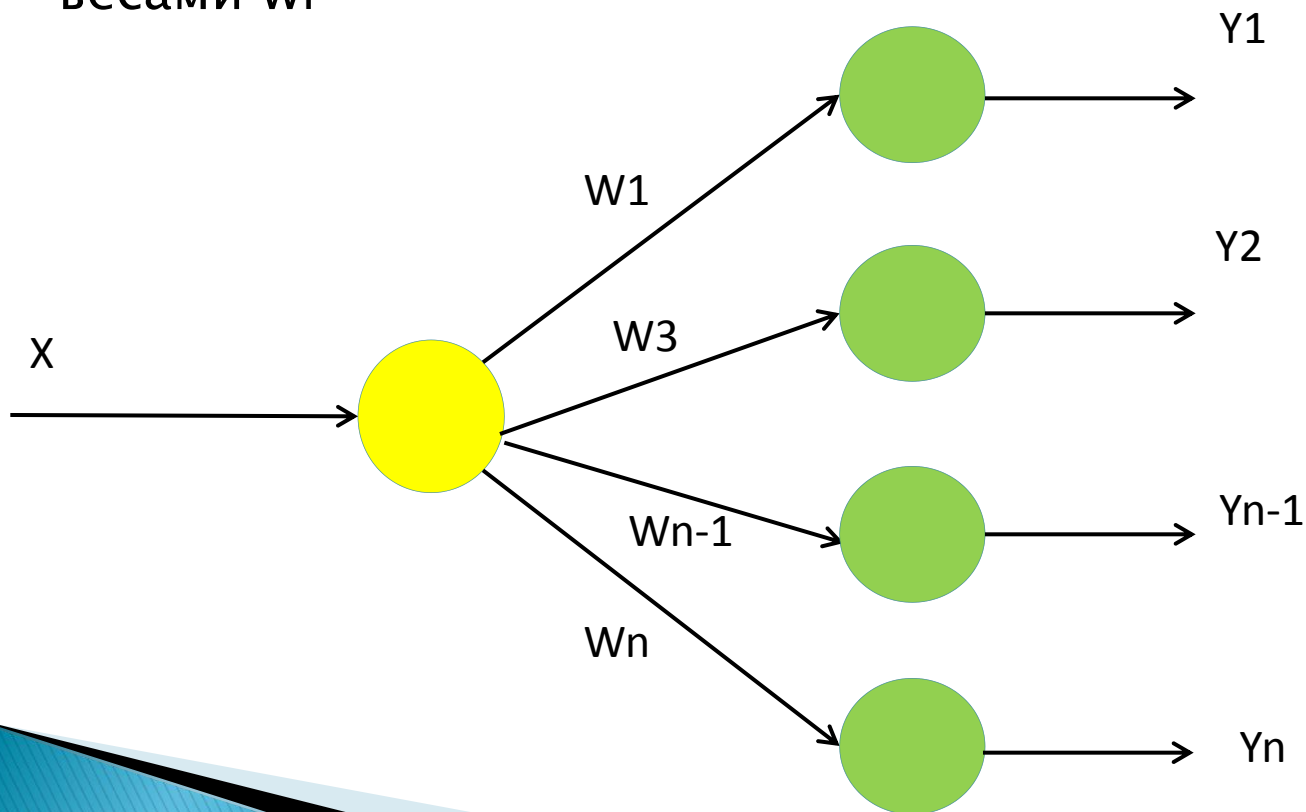
- **Вход:** обучающая выборка, темп обучения α_0 (обычно 0.7 – 0.9)

- **Выход:** веса нейронов Кохонена w_{ij}

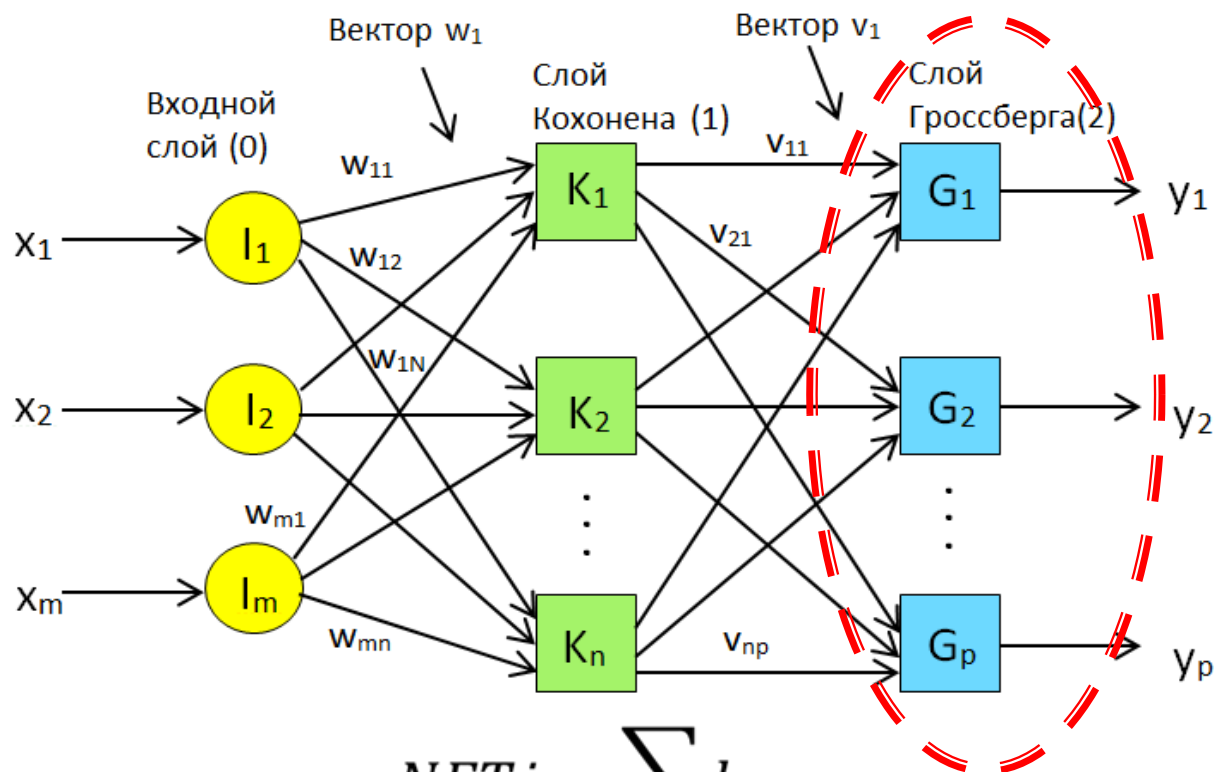


Выходная звезда Гроссберга

- ▶ **Выходная звезда Гроссберга (outstar)** – нейрон с одним входом x и n выходами y_i с весами w_i



Слой Гроссберга



$$NET_j = \sum_i k_i v_{ij},$$

где NET_j – это выход NET нейрона Гроссберга j , или в векторной записи,

$$Y = KV,$$

- где Y – выходной вектор слоя Гроссберга,
- K – выходной вектор слоя Кохонена,
- V – матрица весов слоя Гроссберга

Обучение Full CPN

Благодаря хорошей обобщающей способности двунаправленная сеть встречного распространения способна выдавать верный выход, даже для неполного (поврежденного) входного сигнала.

Обучение full CPN проводится также, как и однонаправленной сети, только теперь вход и выход – это «большой» вектор, составленный из x и y .

Если на вход обученной сети подать только x , то на выходе будут получены аппроксимации для x и y . Если нам известен y , то подав его на вход, получим x и y , т.е. сеть реализует прямое и обратное отображение.

Достоинства CPN: быстро и просто обучается;

Недостатки: менее точная аппроксимация, чем у backprop.

Сети радиальных базисных функций (Powell, 1985)

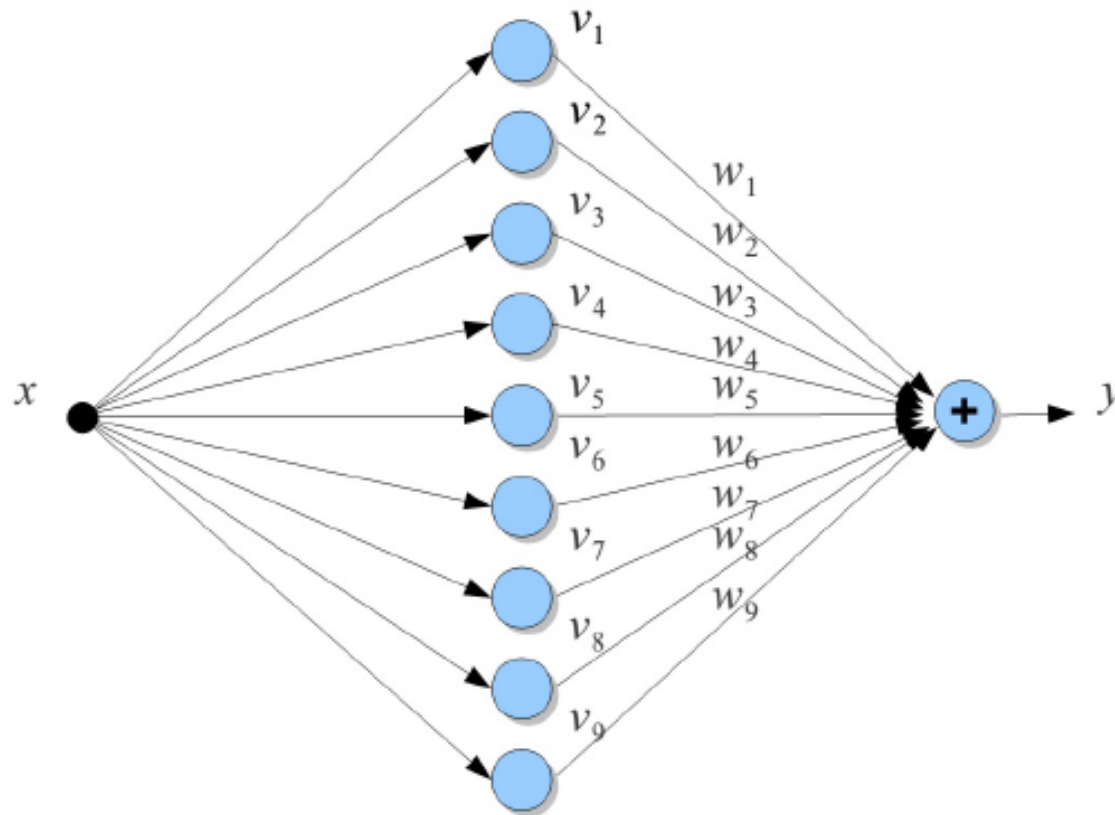
Radial Basis Function Network (RBF) –
в качестве функции активации используют радиальные
функции, чаще всего функцию Гаусса:

$$\varphi(r) = e^{-\frac{r^2}{2\sigma^2}}, \quad \sigma > 0, r \in R.$$

RBF-сети состоят из одного скрытого слоя с радиальными
нейронами и выходного слоя с линейными нейронами.

Структура RBF сети

Сеть 1-9-1 имеет 1 вход, 9 радиальных базисных нейронов, взвешенные выходы которых формируют выход сети.



Обучение RBF сети

Пусть $X, Y \in R^{1 \times N}$ – обучающая выборка.

Каждый радиальный нейрон рассчитывает значение:

$$\varphi_j(x) = e^{-\frac{\|x - c_j\|^2}{2\sigma_j^2}}.$$

После взвешенного суммирования на выходном слое будет:

$$\sum_{j=1}^K w_j \varphi_j(x).$$

Весовые коэффициенты w_j должны обеспечивать равенство этой суммы значению y .

Тогда веса могут быть найдены как решение системы уравнений:

$$\sum_{j=1}^K w_j \varphi_j(x_i) = y_i, \quad i = 1, \dots, N.$$

Обучение RBF сети

В матричном виде система имеет вид:

$$Vw = f,$$

где

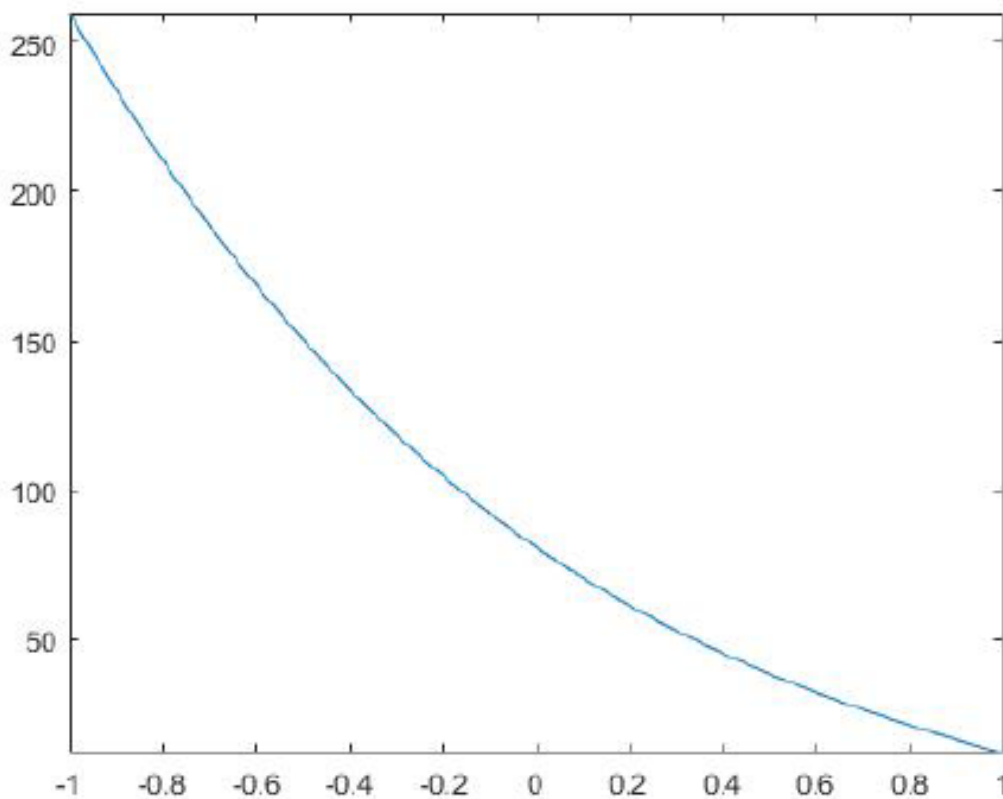
$$V = \begin{pmatrix} \varphi_1(x_1) & \varphi_2(x_1) & \dots & \varphi_K(x_1) \\ \varphi_1(x_2) & \varphi_2(x_2) & \dots & \varphi_K(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_1(x_N) & \varphi_2(x_N) & \dots & \varphi_K(x_N) \end{pmatrix}, w = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_K \end{pmatrix}, f = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

Ясно, что $N \geq K$. При $N = K$ веса находятся с помощью обратной матрицы:

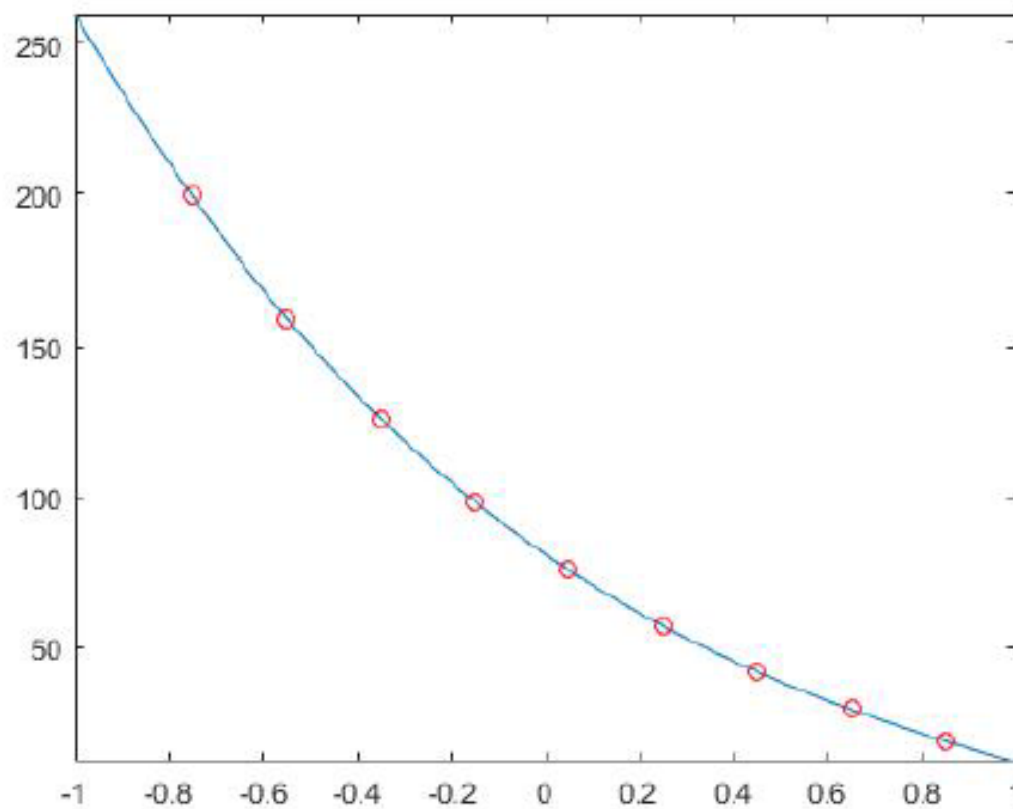
$$w = V^{-1}f.$$

Пример:

Используем RBF сеть для аппроксимации функции $y = (x - 3)^4 - 4x^3$ на интервале $[-1, 1]$.

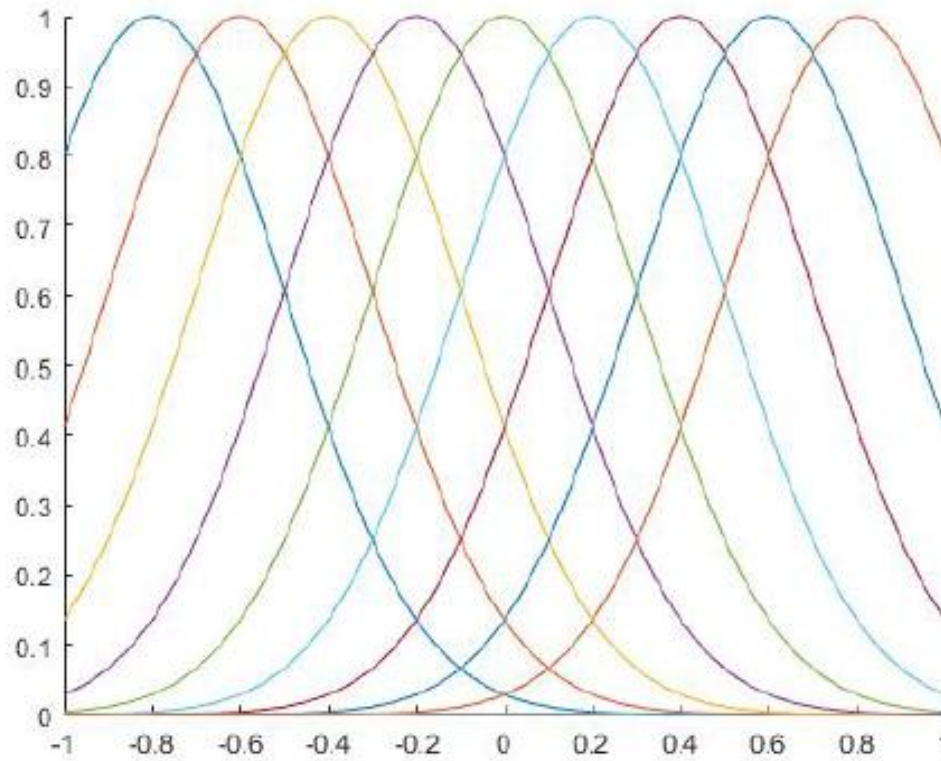


Обучающая выборка: $x = -0.75 : 0.2 : 0.85$, $y = f(x)$.



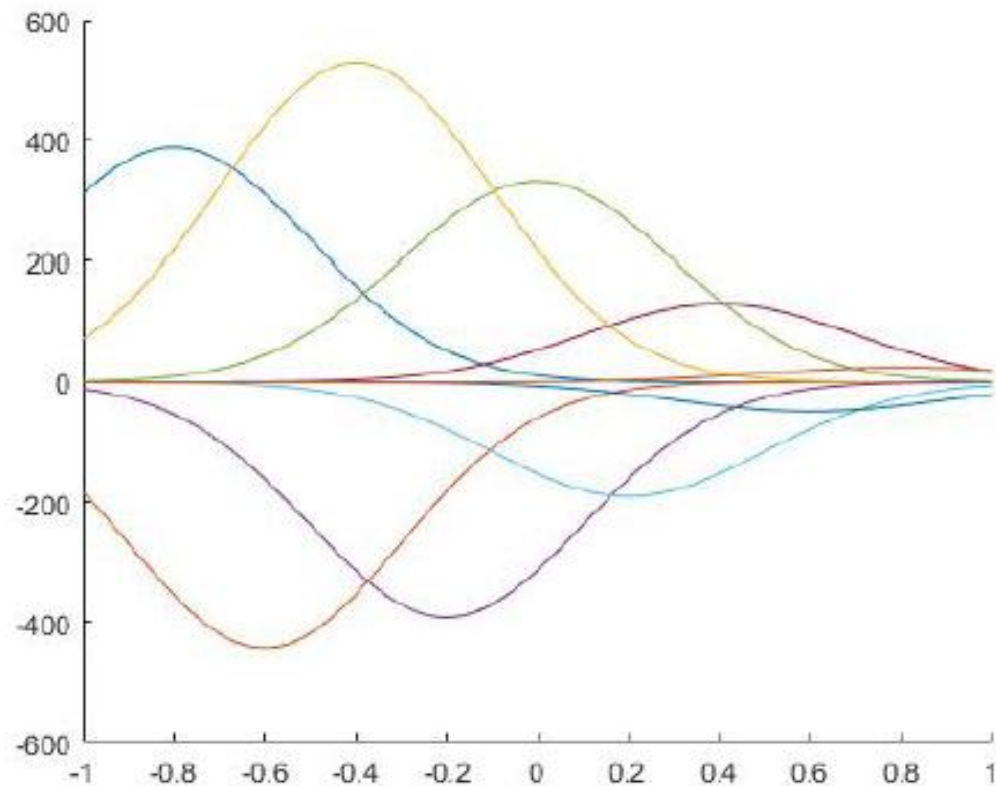
Радиальные функции

Возьмем 9 функций Гаусса: $\varphi_j = e^{-\frac{(x-c_j)^2}{2\sigma^2}}$, $\sigma = 0.3$, центры функций $c = -0.8 : 0.2 : 0.8$.



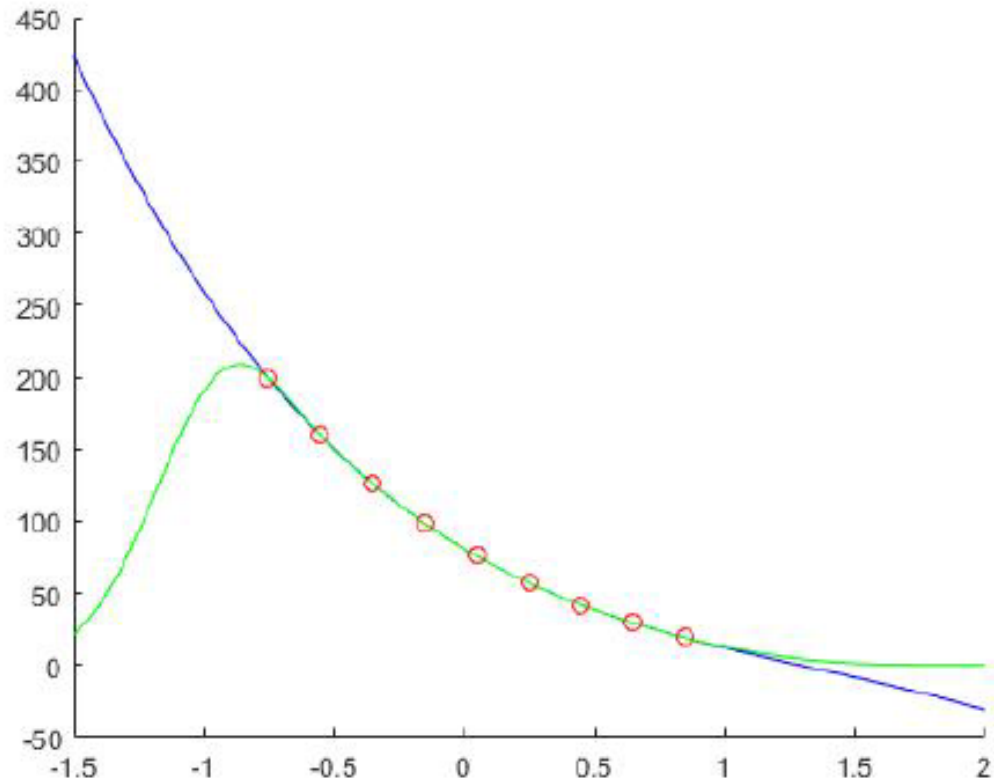
Веса RBF-сети

Найдем веса сети: $w = V^{-1}f$ и масштабируем каждую базисную функцию, умножив на соответствующий вес:



Аппроксимация функции на $[-1; 1]$

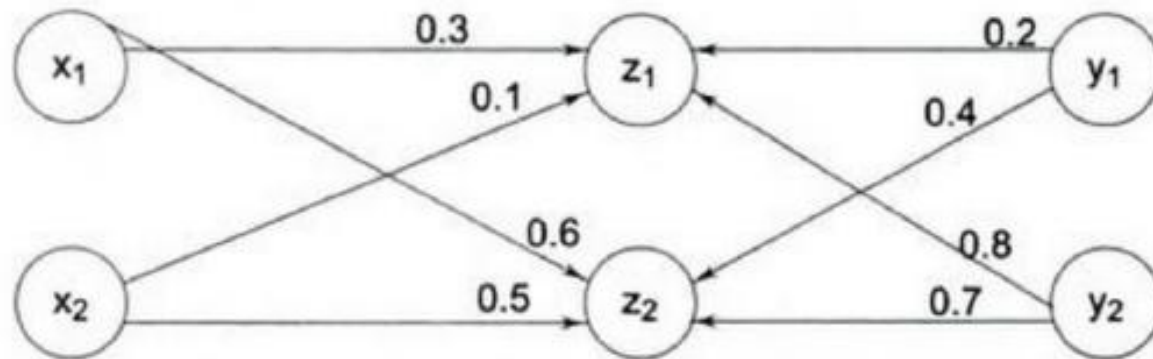
Сумма взвешенных базисных функций и аппроксимируемая кривая построены на рисунке.



Задание

На рисунке представлена однонаправленная сеть встречного распространения.

Для элемента обучающей выборки: $x = (1 \ 0)$, $y = (0 \ 1)$ проведите две итерации обучения слоя Кохонена ($\alpha = 0.5$).



Найдите выход сети. Скорректируйте веса слоя Гроссберга ($\beta = 0.1$).